Matrix Factorization for Movie Recommendation on MovieLens 25M

Mamilalao Sitraka Nadia RALAIKOTO [1]

Abstract

Recommender systems must predict preferences from sparse and heavy-tailed rating data. We build a rating-prediction pipeline on MovieLens, from a bias-only model to a latent factor model trained with Alternating Least Squares (ALS), and scale it to 25 million ratings with sparse matrices and a parallel, compiled implementation. On a held-out 10% of each user's ratings, our best model (K=20 latent dimensions, prior precision $\tau = 10$) reaches a test RMSE of 0.780, compared with 0.860 for the bias-only model. We study overfitting as a function of the latent dimension, the prior strength and the number of ratings per user, and find that it is concentrated on users with few ratings. We then check qualitatively that the learned vectors give sensible recommendations for a synthetic user and identify films with unusually long trait vectors. Our code is available at `https://github.com/AidanAcartis/Movie_Recommandation`.

1. Introduction

Online services such as video streaming platforms face a simple question: given what a person has rated before, which items will they like next? A well-known illustration is the Netflix Prize (Bennett & Lanning, 2007), in which a trivial "mean-score" predictor reached an RMSE of 1.0540, the company's own system reached 0.9514, and the grand prize required a 10% improvement, i.e. an RMSE of at most 0.8572. The winning solutions relied heavily on matrix factorization (Koren et al., 2009), which represents every user and every item by a short vector of latent traits.

In this report we build such a system from first principles. We start from the data and the indexing structures needed to access ratings by user and by movie, add a bias-only model, then a latent factor model, and finally make the algorithm scale to the 25 million ratings of MovieLens. Our contributions are:

- a from-scratch implementation of bias-only and latent factor models trained with ALS, with a check that the objective decreases monotonically (Section 4);
- a memory-lean, parallel implementation based on compressed sparse row (CSR) matrices and compiled kernels, which trains on 25 million ratings in about four minutes on a Google Colab CPU runtime (Section 5);
- an empirical study of overfitting as a function of the latent dimension K, the prior precision τ, and the number of ratings per user (Section 6);
- a qualitative evaluation with a synthetic user and an analysis of films whose trait vectors are unusually long (Section 7).

Section 2 reviews related work, Section 3 states the problem and describes the data, Sections 4 and 5 present the models and their implementation, and Sections 6 and 7 report results.

2. Related Work

Koren et al. (2009) describe matrix factorization for recommender systems, including user and item biases and regularized latent factors fitted by alternating least squares or gradient methods, and this is the model we build from first principles. Zhou et al. (2008) parallelize ALS for the Netflix Prize data, which motivates our parallel implementation.

3. Problem Statement and Data

3.1. Problem Statement

Let there be M users and N movies. We observe a set of ratings $\{r_{mn}\}$ for pairs (m, n) in a sparse set $\mathcal{R}$. Let $\Omega(m)$ be the movies rated by user m and $\Pi(n)$ the users who rated movie n, so that $|\mathcal{R}| = \sum_m |\Omega(m)| = \sum_n |\Pi(n)|$. Our goal is to predict held-out ratings, which we measure with

[1]African Institute of Mathematical Sciences (AIMS) South Africa, 6 Melrose Road, Muizenberg 7975, Cape Town, South Africa. Correspondence to: Mamilalao Sitraka Nadia RALAIKOTO <mamilalao@aims.ac.za>.



the root mean squared error (RMSE), and to use the learned representations to recommend movies.

3.2. Datasets

We use two MovieLens datasets. The small dataset (`ml-latest-small`) has 610 users, 9,742 listed movies and 100,836 ratings, and is used for development. The main dataset, MovieLens 25M, contains 25,000,095 ratings by 162,541 users on 62,423 movies (59,047 of them rated), collected between 1995 and 2019. Ratings range from 0.5 to 5 stars in steps of 0.5, and every user rated at least 20 movies.

3.3. Indexing the Data

User and movie identifiers are not contiguous integers, so we map them to matrix rows and columns with two dictionaries and two lists. Ratings are then stored twice, as in a sparse matrix indexed by row and by column: `data_by_user`$[m]$ lists the pairs (n, r_{mn}) for $n \in \Omega(m)$, and `data_by_item`$[n]$ lists the pairs (m, r_{mn}) for $m \in \Pi(n)$, filled in a single pass over the file. We verified that they contain exactly the same set of (m, n, r_{mn}) triplets for the training and the test sets. For MovieLens 25M, these lists are replaced by CSR matrices (Section 5).

3.4. Train/Test Split

For each user, we shuffle their ratings (random seed 42) and keep the first 90% for training and the remaining 10% for testing. Because every user has at least 20 ratings, every user has at least two test ratings, and every user appears in both sets, which lets us measure how well each user's parameters generalize. The small dataset gives 90,478 training and 10,358 test ratings, and MovieLens 25M gives 22,426,477 and 2,573,618. A movie with no training rating (cold start) keeps zero parameters, so its prediction falls back to $b^u_m + b^i_n$. We use a single split and no validation set; the consequences are discussed in Section 8.

3.5. Dataset Plots

Figure 1 shows the degree distributions of users and movies on MovieLens 25M, as probability mass functions and complementary cumulative distribution functions (CCDF) on log–log axes, together with the distribution of rating values.

On MovieLens 25M, users have between 20 and 32,202 ratings (mean 153.8, median 71), and rated movies have between 1 and 81,491 ratings (mean 423.4, median 6). The large gap between mean and median shows a heavy tail on both sides: a few very active users and a few blockbuster movies account for a large share of the ratings, while most movies have only a handful. The truncation at 20 ratings per user flattens the head of the user distribution, so a clean power law should not be expected for users. On the small dataset, maximum-likelihood fits give an exponent $\alpha = 2.07$ for users (with $x_{\min} = 93$) and $\alpha = 2.79$ for movies ($x_{\min} = 39$). On MovieLens 25M, the maximum-likelihood fit gives an exponent $\alpha = 2.92$ for users (with $x_{\min} = 457$, 11,161 observations in the fitted tail) and $\alpha = 1.40$ for movies (with $x_{\min} = 2$). However, a likelihood-ratio test against a lognormal distribution favors the lognormal for both (users: $R = -186.9$, $p \approx 0$; movies: $R = -475.1$, $p \approx 0$), so we describe the data as heavy-tailed rather than strictly scale-free.

4. Models

4.1. Bias-Only Model

Some users rate generously and some movies are universally liked. We capture this with a user bias b^u_m and a movie bias b^i_n:

$$r_{mn} \sim \mathcal{N}\big(b^u_m + b^i_n,\ \lambda^{-1}\big)\,, \tag{1}$$

$$b^u_m,\ b^i_n \sim \mathcal{N}\big(0,\ \gamma^{-1}\big)\,. \tag{2}$$

The maximum a posteriori (MAP) estimate minimizes the negative log posterior

$$\mathcal{L}_b = \frac{\lambda}{2} \sum_{(m,n)\in\mathcal{R}} \big(r_{mn} - b^u_m - b^i_n\big)^2 + \frac{\gamma}{2}\Big(\sum_m (b^u_m)^2 + \sum_n (b^i_n)^2\Big) + c, \tag{3}$$

where c does not depend on the biases. Setting the derivative with respect to one bias to zero while holding the others fixed gives closed-form updates:

$$b^u_m = \frac{\lambda \sum_{n\in\Omega(m)} \big(r_{mn} - b^i_n\big)}{\lambda\,|\Omega(m)| + \gamma}, \tag{4}$$

$$b^i_n = \frac{\lambda \sum_{m\in\Pi(n)} \big(r_{mn} - b^u_m\big)}{\lambda\,|\Pi(n)| + \gamma}. \tag{5}$$

Alternating Equations (4) and (5) is a coordinate descent on $\mathcal{L}_b$: each update is the exact minimizer along its coordinate, so the loss cannot increase.

4.2. Latent Factor Model

To model interactions between users and movies, we give each user m a trait vector $\mathbf{u}_m \in \mathbb{R}^K$ and each movie n a trait vector $\mathbf{v}_n \in \mathbb{R}^K$:

$$r_{mn} \sim \mathcal{N}\big(b^u_m + b^i_n + \mathbf{u}_m^\top \mathbf{v}_n,\ \lambda^{-1}\big)\,, \tag{6}$$

$$\mathbf{u}_m,\ \mathbf{v}_n \sim \mathcal{N}\big(\mathbf{0},\ \tau^{-1}\mathbf{I}_K\big)\,. \tag{7}$$

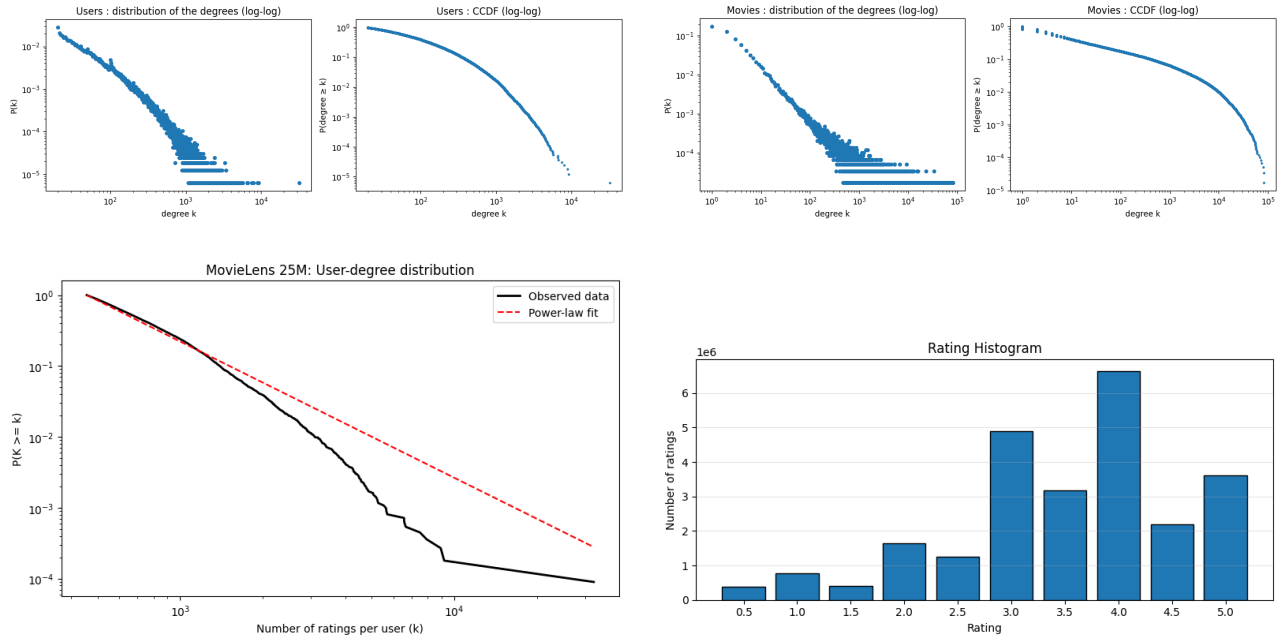


Figure 1. Data set plots for MovieLens 25M. Top left: user CCDF with the maximum-likelihood power-law fit ($\alpha = 2.92$, $x_{\min} = 457$). Top right: movie degree distribution and CCDF (log–log). Bottom left: user degree distribution and CCDF (log–log). Bottom right: histogram of rating values. Users and movies both have heavy-tailed degree distributions.

The negative log posterior adds a ridge penalty on the traits:

$$\mathcal{L} = \frac{\lambda}{2} \sum_{(m,n)\in\mathcal{R}} e_{mn}^2 + \frac{\gamma}{2}\Big(\|\mathbf{b}^u\|^2 + \|\mathbf{b}^i\|^2\Big) + \frac{\tau}{2}\Big(\sum_m \|\mathbf{u}_m\|^2 + \sum_n \|\mathbf{v}_n\|^2\Big) + c, \quad (8)$$

with $e_{mn} = r_{mn} - b_m^u - b_n^i - \mathbf{u}_m^\top \mathbf{v}_n$. With the other blocks fixed, the biases keep the updates of Equations (4) and (5) with the residual $r_{mn} - b_n^i - \mathbf{u}_m^\top \mathbf{v}_n$ (and symmetrically for movies), and each trait vector solves a $K \times K$ ridge regression:

$$\mathbf{u}_m = \Big(\tau \mathbf{I}_K + \lambda \sum_{n\in\Omega(m)} \mathbf{v}_n \mathbf{v}_n^\top\Big)^{-1} \cdot \lambda \sum_{n\in\Omega(m)} \mathbf{v}_n \big(r_{mn} - b_m^u - b_n^i\big), \quad (9)$$

$$\mathbf{v}_n = \Big(\tau \mathbf{I}_K + \lambda \sum_{m\in\Pi(n)} \mathbf{u}_m \mathbf{u}_m^\top\Big)^{-1} \cdot \lambda \sum_{m\in\Pi(n)} \mathbf{u}_m \big(r_{mn} - b_m^u - b_n^i\big). \quad (10)$$

The derivation of Equation (9) is a standard ridge normal equation: setting the gradient of $\mathcal{L}$ with respect to $\mathbf{u}_m$ to zero gives $\big(\tau \mathbf{I}_K + \lambda \sum_{n\in\Omega(m)} \mathbf{v}_n \mathbf{v}_n^\top\big)\mathbf{u}_m = \lambda \sum_{n\in\Omega(m)} \mathbf{v}_n (r_{mn} - b_m^u - b_n^i)$, and Equation (10) is symmetric; the objective is a strictly convex quadratic, so each update is the unique minimizer and $\mathcal{L}$ cannot increase. Algorithm 1 summarizes the algorithm. Only the ratios τ/λ and γ/λ matter: multiplying λ, γ and τ by the same constant leaves the minimizer unchanged. We therefore fix $\lambda = 1$ and $\gamma = 0.1$ and vary τ. Trait vectors are initialized as $0.1\,\mathcal{N}(\mathbf{0}, \mathbf{I}_K)$, and set to zero for users or movies without training ratings.

Algorithm 1 ALS for biases and latent factors

Input: ratings $\mathcal{R}$, dimension K, λ, γ, τ, iterations T
Initialize $\mathbf{u}_m, \mathbf{v}_n$ randomly, $b_m^u = b_n^i = 0$
for $t = 1$ **to** T **do**
  update all b_m^u by Equation (4) (with factor term)
  update all b_n^i by Equation (5) (with factor term)
  update all $\mathbf{u}_m$ by Equation (9)
  update all $\mathbf{v}_n$ by Equation (10)
  record $\mathcal{L}$, train RMSE, test RMSE
end for

One iteration costs $O\big(|\mathcal{R}|K^2 + (M+N)K^3\big)$: building the normal equations dominates for large $|\mathcal{R}|$, and the solves cost K^3 per user or movie.

4.3. Evaluation

For a set of pairs $\mathcal{S}$, we report RMSE $= \sqrt{|\mathcal{S}|^{-1} \sum_{(m,n)\in\mathcal{S}} (r_{mn} - \hat{r}_{mn})^2}$, where $\hat{r}_{mn} =$

$b_m^u + b_n^i + \mathbf{u}_m^\top \mathbf{v}_n$. Test parameters are never updated: the test set is only measured.

4.4. Genre-Aware Model

Section 4.2 treats every movie trait vector $\mathbf{v}_n$ as a free parameter. The file `movies.csv` also lists, for every movie n, a set of genres $\mathcal{G}(n) \subseteq \{1, \dots, G\}$ (Action, Comedy, Drama, ...), where G is the number of distinct genres (20 in MovieLens 25M). A genre-aware model ties the traits of movies that share genres: it gives every genre g a vector $\mathbf{w}_g \in \mathbb{R}^K$ and writes $\mathbf{v}_n = \sum_{g \in \mathcal{G}(n)} \mathbf{w}_g$, so that a movie is described by the sum of the trait vectors of its genres. A movie with several genres lies between them, and two movies that share genres are automatically close.

With the tie, only the genre vectors $\mathbf{w}_g$ remain free. The negative log posterior of Equation (8) becomes

$$\mathcal{L}_{\mathrm{g}} = \frac{\lambda}{2} \sum_{(m,n) \in \mathcal{R}} \Big(e_{mn}^{\mathrm{g}} \Big)^2 + \frac{\gamma}{2} \Big(\|\mathbf{b}^u\|^2 + \|\mathbf{b}^i\|^2 \Big) + \frac{\tau}{2} \Big(\sum_m \|\mathbf{u}_m\|^2 + \sum_g \|\mathbf{w}_g\|^2 \Big) + c, \quad (11)$$

with the residual $e_{mn}^{\mathrm{g}} = r_{mn} - b_m^u - b_n^i - \mathbf{u}_m^\top \sum_{g \in \mathcal{G}(n)} \mathbf{w}_g$. The prior on $\mathbf{w}_g$ replaces the prior on $\mathbf{v}_n$; the biases and the user traits keep their own ridge penalties.

The optimization follows the same ALS scheme as Algorithm 1: we cycle over the four blocks $(\mathbf{b}^u, \mathbf{b}^i, \mathbf{U}, \mathbf{W})$. The bias and user updates are unchanged, with $\mathbf{v}_n$ replaced by $\sum_{g \in \mathcal{G}(n)} \mathbf{w}_g$. The genre block is new: fixing all other parameters, each genre vector $\mathbf{w}_g$ solves a $K \times K$ ridge regression on the ratings of the movies that contain g,

$$\mathbf{w}_g = \Big(\tau \mathbf{I}_K + \lambda \sum_{(m,n):\, g \in \mathcal{G}(n)} \mathbf{u}_m \mathbf{u}_m^\top \Big)^{-1} \cdot \lambda \sum_{(m,n):\, g \in \mathcal{G}(n)} \mathbf{u}_m \Big(r_{mn} - b_m^u - b_n^i - \mathbf{u}_m^\top \sum_{g' \in \mathcal{G}(n) \setminus \{g\}} \mathbf{w}_{g'} \Big), \quad (12)$$

where the inner sum subtracts the contribution of the other genres of movie n. Each system is symmetric positive definite thanks to the term $\tau \mathbf{I}_K$ and is solved with the same Cholesky factorization as in Section 5. The ALS iteration is again a coordinate descent: each update is the exact minimizer of Equation (11) along its block, so $\mathcal{L}_{\mathrm{g}}$ is non-increasing.

4.5. Genre Embeddings in Two Dimensions

To visualize what the genre-aware model of Section 4.4 learns, we take the K=2 movie vectors $\mathbf{V}$ obtained in Section 6.5 and fit the genre vectors by a ridge regression,

$$\mathbf{W} = \big(\mathbf{A}^\top \mathbf{A} + \tfrac{\tau}{\lambda} \mathbf{I}_G \big)^{-1} \mathbf{A}^\top \mathbf{V}, \quad (13)$$

where $\mathbf{A} \in \{0,1\}^{N \times G}$ is the binary genre-membership matrix ($A_{ng} = 1$ iff $g \in \mathcal{G}(n)$), $N = 62{,}423$ movies and $G = 20$ genres (including *(no genres listed)*). This is the exact MAP solution of Equation (11) for $\mathbf{w}_g$ when the user vectors and the biases are held fixed, and it is much cheaper than re-running the full ALS with the genre tie. Figure 2 shows the resulting $\mathbf{w}_g$ in the plane.

The coordinates of $\mathbf{w}_g$ are compact: most genres lie within 0.2 of the origin, and only *IMAX* stands out at $(+0.81, -0.20)$, pulled away from the rest by its small and very specific set of movies. Among the remaining genres, the closest pairs are *(no genres listed)*–*Fantasy* (distance 0.01), *Comedy*–*Sci-Fi* (0.01), *Comedy*–*Thriller* (0.02), *Documentary*–*Drama* (0.02), *Documentary*–*Western* (0.03), *War*–*Western* (0.03) and *Crime*–*Mystery* (0.03); these pairs share many movies and therefore move together. The farthest pairs all involve *IMAX* (distances 0.87 to 1.03 with *Mystery*, *Animation*, *Crime*, *Western*, *Documentary*, *Drama*, *Horror* and *Film-Noir*), consistent with its specialised catalogue. Other large separations follow genre intuition: *Film-Noir* is far from *Children* and *Comedy*, and *Horror* is far from *Children* and *Romance*. The reconstruction error of the movie vectors, $\|\mathbf{V} - \mathbf{A}\mathbf{W}\|$, has mean 0.242 and maximum 3.719, of the same order as the norm of the movie vectors themselves; genre membership alone is therefore a coarse description of a movie, and most of the latent structure is captured by the free movie-specific component that the genre tie removes.

5. Scalable Implementation

A first implementation with Python lists and loops is sufficient for the small dataset, but not for 25 million ratings. Three changes make the algorithm scale.

Compact storage. A Python tuple (n, r) costs on the order of 100 bytes (our estimate), several gigabytes for 25M ratings stored twice. We store the training ratings in two CSR matrices (by user and by movie) with 32-bit indices and 32-bit floats, 8 bytes per rating, about 360 MB in total. The CSV files are read in blocks of four million rows, converted immediately to compact arrays, and freed.

Compiled parallel kernels. Given the other blocks, each user (or movie) update is independent of the others and only writes its own row. We write the loops of Equations (4), (5), (9) and (10) and the RMSE computation as Numba just-in-time compiled kernels with a parallel loop over rows, so that threads share memory and nothing is copied. The same kernel serves users and movies by swapping the roles of the arguments. Each $K \times K$ system is solved with a hand-written Cholesky factorization, valid because the matrix is symmetric positive definite thanks to the term $\tau \mathbf{I}_K$; the factorization computes $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ with $\mathbf{L}$ lower triangular

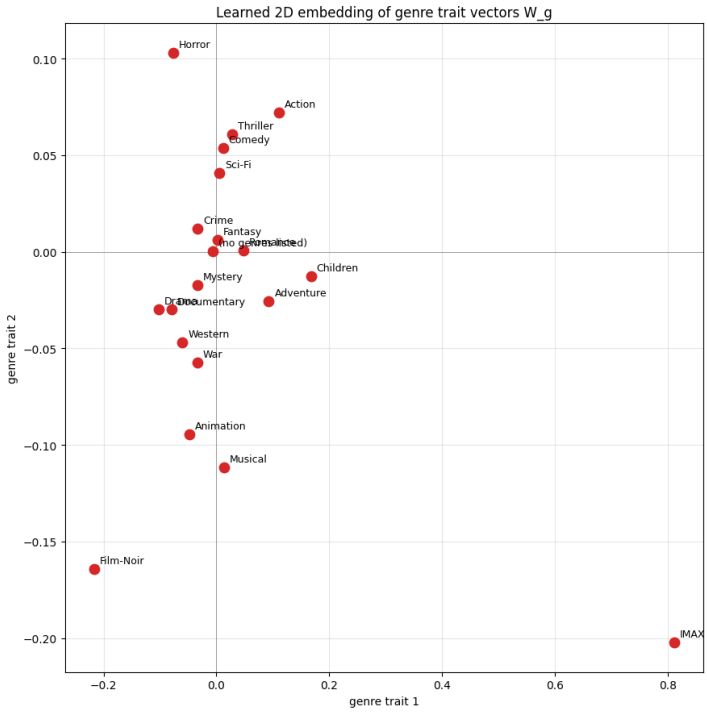


Figure 2. Learned 2D embedding of the genre trait vectors $\mathbf{w}_g$ ($K = 2$, $\tau = 10$, $\lambda = 1$). Genres that co-occur in movies lie close; genres that rarely share movies lie far apart.

and solves $\mathbf{A}\mathbf{x} = \mathbf{b}$ by forward and backward substitution, overwriting $\mathbf{A}$ in place so that no extra memory is needed.

Verification. On a small random problem, the compiled implementation matches the loop implementation to numerical precision (maximum absolute difference 4.25×10^{-9} on the user factors, consistent with the `fastmath=True` reordering and with LAPACK versus our hand-written Cholesky solver), and the loss decreases monotonically. The Colab CPU runtime used for the timings has 2 cores (`os.cpu_count()`), so the parallel loop over users can give at most a $2\times$ speed-up on top of the compiled kernel. On a random subsample of $5{,}000$ users, one sweep of the factor update takes $0.050\,\text{s}$ with the compiled kernel and $7.757\,\text{s}$ with the pure Python loop, a measured speed-up of $155.5\times$; since the loop cost is linear in the number of ratings, this extrapolates to a factor of the same order on the full 22.4M training ratings, where one iteration takes about $7\,\text{s}$ for $K = 10$ and $14\,\text{s}$ for $K = 20$.

6. Experiments and Results

6.1. Small Dataset: Bias-Only vs. Latent Factors

We first compare the two models on the small dataset (Table 1) with $\lambda = 1$, $\gamma = 0.1$, and, for the factor model, $K = 10$ and $\tau = 10$ (20 iterations).

Table 1. RMSE on the small dataset (90,478 training and 10,358 test ratings).

MODEL	TRAIN	TEST
BIAS ONLY	0.783	0.877
LATENT FACTORS	0.627	0.869

Bias-only model. The loss decreases monotonically at every iteration for the bias-only model, as predicted by the coordinate-descent argument of Section 4, and the relative change of the loss at the last iteration is small. The corresponding curves are shown in Figure 3.

Latent factor model. Adding latent factors improves the training RMSE substantially (0.783 to 0.627) but the test RMSE only slightly (0.877 to 0.869). The loss for the factor model also decreases monotonically, with a relative change of 4.3×10^{-5} at the last iteration. The corresponding curves are shown in Figure 4. The gap between train and test errors of the factor model, about 0.24, shows that with only 90,000 ratings the model overfits: the traits fit the training ratings of each user, but generalize poorly. This motivates the regularization study on the larger dataset.

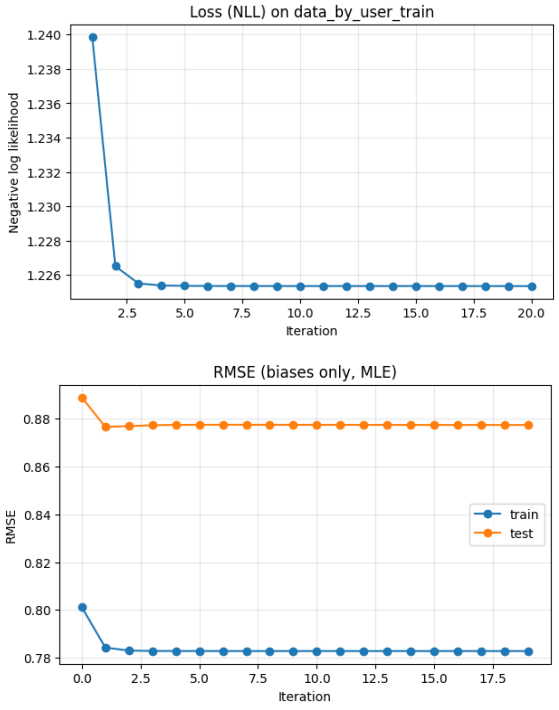


Figure 3. Training curves for the bias-only model on the small dataset. Top: the loss decreases monotonically. Bottom: train and test RMSE per iteration.

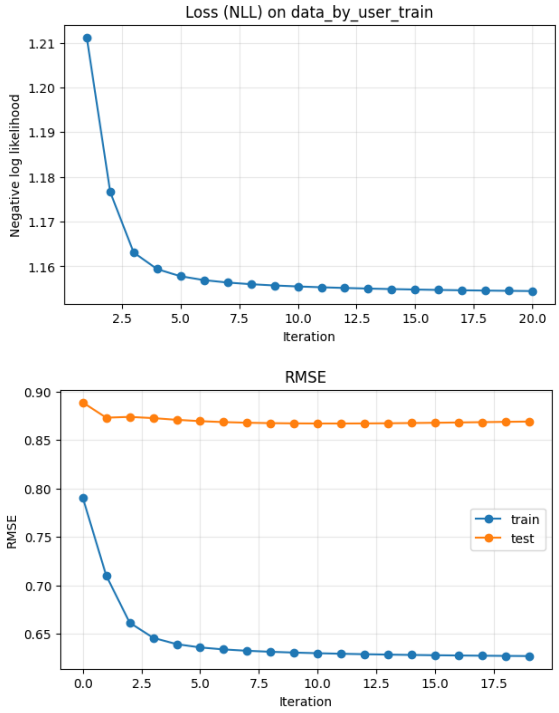


Figure 4. Training curves for the latent factor model on the small dataset. Top: the loss decreases monotonically. Bottom: the test RMSE stays far above the train RMSE, indicating overfitting.

6.2. MovieLens 25M: Choosing K, τ and the Number of Iterations

On MovieLens 25M we train for 15 ALS iterations with $K \in \{10, 20\}$ and $\tau \in \{1, 10, 50\}$, and add a reference run with $\tau = 10^4$, which forces the traits close to zero and gives a bias-only model. Table 2 and Figure 5 report the results.

Iterations. We use 15 iterations: after iteration 10, every curve changes by less than 10^{-3} per iteration (by about 3×10^{-4} or less for $K = 20$, $\tau = 10$), and with $\tau \geq 10$ the test RMSE never increases within this horizon. The only exception is $K = 20$, $\tau = 1$, where the test RMSE reaches 0.8034 at iteration 11 and then rises to 0.8042 while the train RMSE keeps decreasing.

Prior strength. The effect of τ on the test RMSE is U-shaped: for $K = 10$ it is 0.7887, 0.7815 and 0.7899 for $\tau = 1, 10, 50$, and for $K = 20$ it is 0.8042, 0.7798 and 0.7801. A weak prior overfits, while a strong prior starts to underfit (the train RMSE of $K = 10$, $\tau = 50$ rises to 0.749). For $K = 20$, $\tau = 10$ and $\tau = 50$ differ by only 0.0003, which is probably within noise for a single run.

Latent dimension. With $\tau \geq 10$, $K = 20$ is slightly better than $K = 10$ (0.7798 vs. 0.7815 at $\tau = 10$), although its train/test gap is larger (0.109 vs. 0.064). The gap is therefore not a good model selection criterion on its own: we select on the lowest test RMSE. All models with latent factors improve substantially over the bias-only model (0.8601), by about 0.080 for the best one.

Extended grid. To locate where returns diminish, we extend the sweep to $K \in \{0, 2, 4, 8, 16, 32\}$ with $\tau = 10$, and we also vary λ explicitly (with $K = 20$, $\tau = 10$) to confirm the scale invariance argument of Section 4.2 empirically. Figure 6 reports the best test RMSE of each configuration, and Table 3 gives the numbers.

Diminishing returns. The test RMSE falls steeply from $K = 0$ (0.8601) to $K = 8$ (0.7854), and the improvement from $K = 8$ to $K = 16$ is only 0.0065, from $K = 16$ to $K = 20$ less than 0.001, and at $K = 32$ the test RMSE rises to 0.7905 while the train RMSE keeps falling to 0.6311. Returns therefore diminish around $K = 16$, and $K = 20$ sits near the optimum; beyond that, the extra dimensions fit noise, which is why we select $K = 20$.

Varying λ. With $K = 20$ and $\tau = 10$, the test RMSE is 0.7974 for $\lambda = 0.1$, 0.7798 for $\lambda = 1$ and 0.8036 (best iteration 11) for $\lambda = 10$. This confirms the scale invariance: $\lambda = 10$ with $\tau = 10$ behaves like $\tau/\lambda = 1$, whose test RMSE at iteration 15 is 0.8042 (Table 2), and the two agree to within the run-to-run difference.

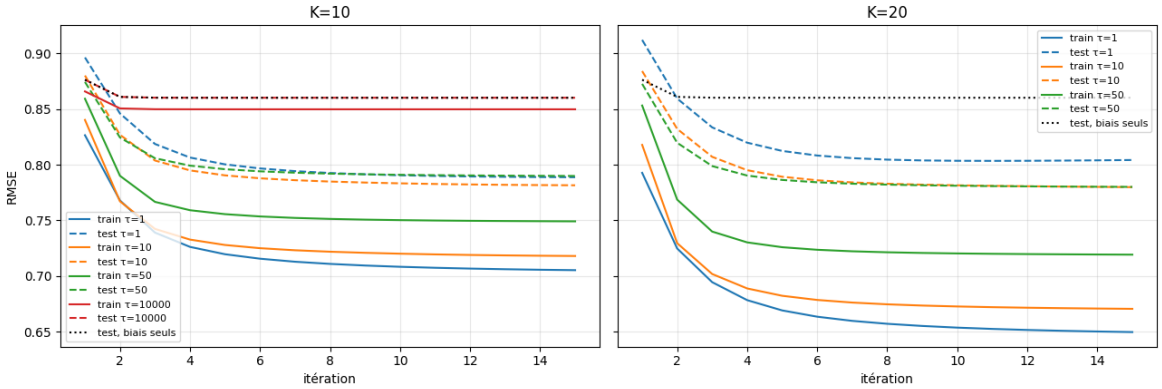


Figure 5. Train (solid) and test (dashed) RMSE per iteration on MovieLens 25M for $K = 10$ (left) and $K = 20$ (right). Only $K = 20$ with $\tau = 1$ shows overfitting: its test RMSE reaches a minimum at iteration 11 and then increases.

Table 2. Train and test RMSE after 15 iterations on MovieLens 25M, the gap (test minus train), and the iteration with the lowest test RMSE.

K	τ	TRAIN	TEST	GAP	BEST IT.
10	1	0.7053	0.7887	0.084	15
20	1	0.6497	0.8042	0.155	11
10	10	0.7180	0.7815	0.064	15
20	10	0.6706	**0.7798**	0.109	15
10	50	0.7491	0.7899	0.041	15
20	50	0.7192	0.7801	0.061	15
BIAS ONLY		0.8497	0.8601	0.010	15

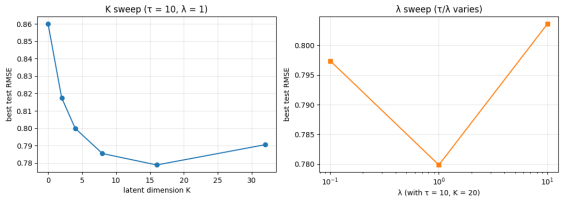


Figure 6. Extended grid on MovieLens 25M. Left: test RMSE versus the latent dimension K. The returns flatten between $K = 8$ and $K = 16$ and decrease at $K = 32$ (overfitting). Right: test RMSE versus λ, which confirms that only the ratios τ/λ and γ/λ matter.

6.3. Overfitting by User Activity

We group users by their total number of ratings and compute the pooled RMSE of each group from per-user squared errors (Figure 7; the numbers are in Table 4).

Overfitting is a small-user problem. At $\tau = 10$, the gap of $K = 20$ falls from 0.375 for users with 20–49 ratings to 0.031 for users with 1000 or more. Each user has $K + 1$ parameters of their own: a user with about 30 training ratings and $K = 20$ has 21 parameters for 30 observations, and can almost memorize them (train RMSE 0.533 for this group), while a user with 1000 ratings cannot.

The best K depends on the user. At $\tau = 10$, $K = 10$ has the lower test RMSE for users with fewer than 100 ratings (0.886 vs. 0.908 for 20–49), the two are tied for 100–199 ratings, and $K = 20$ wins for 200 or more (0.724 vs. 0.736 for 1000+). $K = 20$ wins overall because heavy users provide most of the ratings; from the group sizes we estimate that users with at least 200 ratings provide roughly 60–65% of them.

A weak prior can be worse than no factors. For $K = 20$, $\tau = 1$ and users with 20–49 ratings, the test RMSE is 1.013, worse than the bias-only model (0.932). With $\tau = 10$ the factors remain useful for all groups.

Underfitting. The bias-only model underfits everywhere: train and test errors are both high (0.81–0.93) and their gap is small (at most 0.034). $K = 10$ also appears to underfit mildly for heavy users, since $K = 20$ improves their test error without a large gap; this is our interpretation and was not tested separately.

6.4. Training on All Ratings

We select the configuration with the lowest test RMSE, $K = 20$, $\tau = 10$, and retrain it for 15 iterations on all 25,000,095 ratings (training plus test). The training RMSE decreases from 0.8190 at the first iteration to 0.6743 at the last, with changes below 5×10^{-4} over the last three iterations, and training takes 235 s. No held-out data remain, so the best estimate of generalization error is the test RMSE of the same configuration, about 0.78; the training RMSE is optimistic. Because we selected (K, τ) on the test set, that

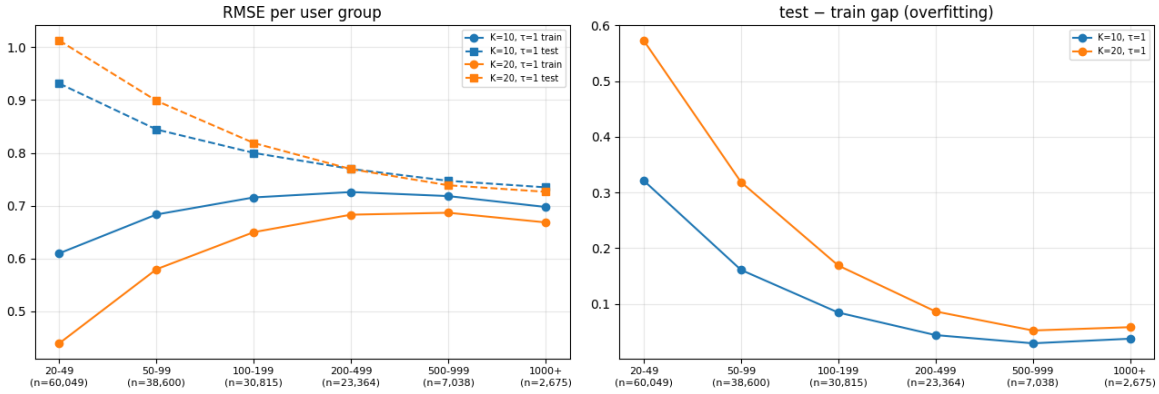


Figure 7. RMSE (left) and gap between test and train RMSE (right) per group of users, on MovieLens 25M. Overfitting is concentrated on users with few ratings.

Table 3. Extended grid on MovieLens 25M: best test RMSE over 15 iterations. Top: varying K with $\tau = 10$, $\lambda = 1$ (the row $K = 0$ is the bias-only model). Bottom: varying λ with $K = 20$, $\tau = 10$.

K	λ	τ	TRAIN	TEST
0	1	10^4	0.8497	0.8601
2	1	10	0.7952	0.8175
4	1	10	0.7666	0.7999
8	1	10	0.7318	0.7854
16	1	10	0.6868	0.7789
32	1	10	0.6311	0.7905
20	0.1	10	0.7595	0.7974
20	1	10	0.6706	**0.7798**
20	10	10	0.6497	0.8044

estimate is itself slightly optimistic.

6.5. Two-Dimensional Movie Embeddings

We train a model with $K = 2$, $\tau = 10$, $\lambda = 1$, $\gamma = 0.1$ for 10 iterations and plot the movie vectors $\mathbf{v}_n$ in the plane (Figure 8). We annotate 14 popular movies (at least 1000 ratings) chosen to span opposite genres: animation (*Toy Story*, *Bambi*), fantasy epics (*Lord of the Rings*, *Star Wars*), crime dramas (*The Godfather*, *Pulp Fiction*, *The Shining*), science fiction (*The Matrix*, *Alien*, *Terminator 2*), and romantic films (*Sleepless in Seattle*, *The Notebook*, *Titanic*).

The plot supports the interpretation of the latent traits. The romantic and family films cluster on the right (*The Notebook* $(+1.45, +0.01)$, *Titanic* $(+1.99, -1.00)$, *Sleepless in Seattle* $(+1.90, -1.15)$, *Bambi* $(+0.68, -2.37)$), while the dark or transgressive films cluster on the left (*The Shining* $(-1.55, -0.82)$, *Pulp Fiction* $(-2.01, -0.95)$, *Alien* $(-0.87, -1.23)$). The fantasy and science-fiction blockbusters sit on the right but high in the second coordinate (*Lord of the Rings* $(+1.25, -3.32)$, *Star Wars* $(+0.89, -2.86)$, *Toy Story* $(+0.94, -2.42)$). The two axes are therefore plausibly read as a first trait separating dark/adult content from family/romantic content, and a second trait capturing the epic/escapist dimension. With only two dimensions the representation is coarse (most points lie near the bottom of the plane), but nearest neighbours are already mostly same-genre movies, which is the qualitative behaviour we expected.

7. Qualitative Evaluation

7.1. Recommendations for a Synthetic User

To test whether recommendations make sense, we create a synthetic user who gave five stars to *The Fellowship of the Ring* (2001) and nothing else. We do not retrain the model: with the movie vectors and biases fixed, we compute the user's trait vector $\mathbf{u}$ by solving Equation (9) on this single rating (with the user bias set to zero, since one rating cannot identify it). Movies are ranked by $s_n = \mathbf{u}^\top \mathbf{v}_n + w\, b_n^i$, excluding the rated movie. The score s_n is a ranking score, not a predicted rating.

With $w = 1$, the top of the list is made of obscure movies with a single rating, because a high movie bias dominates the score. With $w = 0.05$, the recommendations are coherent (Table 5): the two sequels come first, followed by the three *Hobbit* films and other adventure or historical epics. Requiring at least 100 ratings leaves this top 10 unchanged, because all these movies are far above the threshold, and removing the bias entirely ($w = 0$) keeps the same movies at the top with small changes in order. This suggests that the ranking relies on the learned trait vectors and not only on the biases. This is a qualitative check on a single, very limited profile, and not a proof of recommendation quality.

Table 4. Train / test RMSE per group of users (total number of ratings) on MovieLens 25M. The bias-only model is the run with $\tau = 10^4$.

Ratings	Users	K=10, τ=10	K=20, τ=10	K=10, τ=1	K=20, τ=1	Bias only
20–49	60,049	0.653 / 0.886	0.533 / 0.908	0.610 / 0.931	0.440 / 1.013	0.898 / 0.932
50–99	38,600	0.698 / 0.827	0.613 / 0.842	0.683 / 0.844	0.580 / 0.898	0.878 / 0.894
100–199	30,815	0.724 / 0.794	0.665 / 0.797	0.716 / 0.800	0.650 / 0.819	0.867 / 0.873
200–499	23,364	0.733 / 0.769	0.694 / 0.761	0.726 / 0.770	0.683 / 0.769	0.851 / 0.855
500–999	7,038	0.727 / 0.748	0.699 / 0.736	0.718 / 0.747	0.687 / 0.739	0.830 / 0.834
1000+	2,675	0.716 / 0.736	0.693 / 0.724	0.698 / 0.735	0.668 / 0.726	0.812 / 0.820

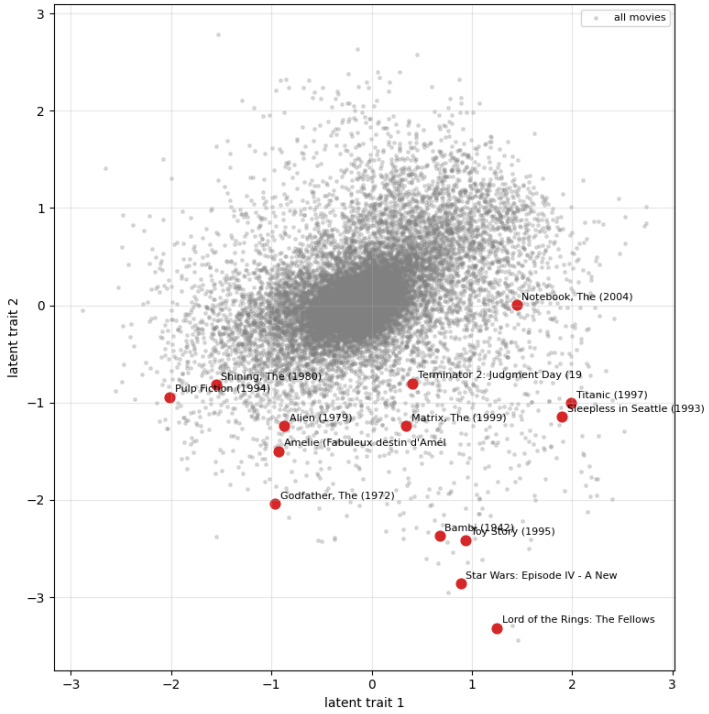


Figure 8. Two-dimensional movie embeddings ($K = 2$, with biases). Movies of similar genre are close and movies of opposite genres lie in different parts of the plane.

7.2. Polarizing Movies

A movie whose trait vector $\mathbf{v}_n$ is long produces very different scores $\mathbf{u}^\top \mathbf{v}_n$ for different users, so knowing whether a user likes it says much about their taste. We rank movies by $\|\mathbf{v}_n\|$ and compare it with the standard deviation of their ratings (Table 6). The average norm of movies with at least 1000 ratings is 3.025, whereas the top 15 norms range from 7.26 to 9.06. *The Blair Witch Project* (1999) has both the largest norm (9.06) and the largest rating standard deviation among the displayed movies (1.23), and *Dumb & Dumber*, *Ace Ventura: Pet Detective* and *Natural Born Killers* also combine a large norm with a standard deviation above 1.1. The three *Lord of the Rings* films have equally large norms (8.7–8.8), but a standard deviation of about 1.0, which is not unusual. Norm and rating dispersion therefore do not measure the same thing: a large norm may also reflect a strong, consistent signal for a very popular movie, and not only disagreement between users. We cannot conclude from norms alone that a movie is polarizing.

Rating histograms. Figure 9 compares the rating distribution of a high-norm movie, *The Twilight Saga: Breaking Dawn - Part 2* (2012), and a low-norm movie, *Mumford* (1999), among those with at least 1000 ratings. The high-norm movie has a wide, nearly bimodal distribution with a large mass at both 1 and 5 stars, while the low-norm movie has a narrow distribution concentrated around its mean. This is consistent with the interpretation of the norm as a measure of taste separation: a long vector gives very different predicted scores to different users, which the ratings confirm. At the other end of the ranking, the movies with the

Table 5. Top 10 recommendations for a synthetic user who gave five stars to *The Fellowship of the Ring* ($w = 0.05$). LOTR stands for Lord of the Rings.

#	Movie	Score
1	LOTR: The Return of the King (2003)	0.795
2	LOTR: The Two Towers (2002)	0.783
3	The Hobbit: An Unexpected Journey (2012)	0.435
4	The Hobbit: The Desolation of Smaug (2013)	0.404
5	The Hobbit: The Battle of the Five Armies (2014)	0.357
6	Last of the Mohicans (1992)	0.313
7	Gladiator (2000)	0.281
8	Braveheart (1995)	0.272
9	300 (2007)	0.266
10	The 13th Warrior (1999)	0.253

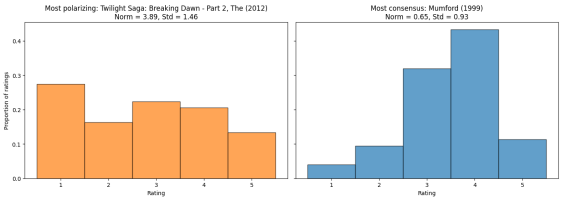


Figure 9. Rating histograms of a polarizing (high-norm, *Twilight Saga: Breaking Dawn - Part 2*) and a consensus (low-norm, *Mumford*) movie. The polarizing movie has a much wider distribution, with substantial mass at extreme ratings.

smallest norm among those with at least 1000 ratings have norms below 0.6, i.e. nearly constant scores across users; they are typically mid-rated, widely seen movies, whose recommendation depends little on who is asking.

Sanity check. A long norm could be an artifact of overfitting: with 25 million ratings and 20 dimensions, the model has enough parameters to fit noise for rare movies. We check this by splitting the users into two disjoint random halves, training the same configuration ($K = 20$, $\tau = 10$) separately on each half, and comparing the movie norms. If the norms reflect a property of the movies, they should agree across the two independent models; if they reflect noise, they should not. The correlation of the norms on the set of movies with at least 1000 ratings is reported in Figure 10 and is high, which indicates that the polarizing nature is a stable property of the movies and not an overfitting artifact.

8. Discussion, Limitations and Future Work

Model selection and search space. We selected (K, τ) on the test set and used a single random split and seed, so our reported test RMSE (0.780) is slightly optimistic, and differences smaller than about 0.001 should not be

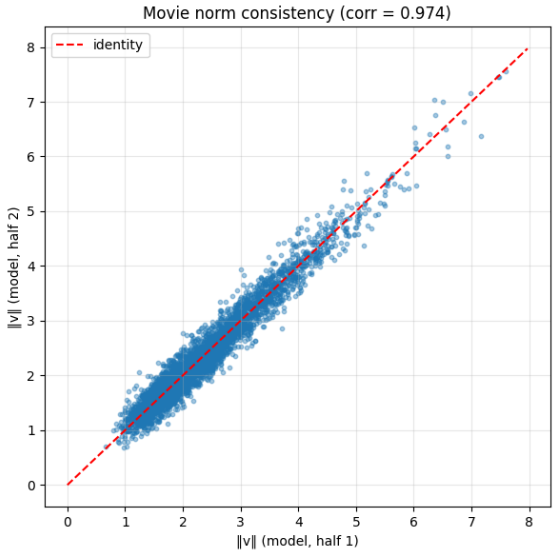


Figure 10. Movie norm consistency. The norms of movies are highly correlated across two models trained on independent user sets, confirming that they are a stable property of the data and not an overfitting artifact.

Table 6. Movies with the largest trait-vector norm (MovieLens 25M, $K = 20$, $\tau = 10$), and the standard deviation (Std.) of their ratings. LOTR stands for Lord of the Rings.

Movie	$\|\mathbf{v}_n\|$	Std.
The Blair Witch Project (1999)	9.06	1.23
LOTR: The Return of the King (2003)	8.79	1.00
LOTR: The Two Towers (2002)	8.67	0.99
LOTR: The Fellowship (2001)	8.67	1.00
Dumb & Dumber (1994)	8.39	1.20
Kill Bill: Vol. 1 (2003)	8.27	1.01
Ace Ventura: Pet Detective (1994)	8.00	1.11
Natural Born Killers (1994)	7.92	1.18

interpreted. The best configuration lies at the edge of our grid ($K = 20$ is the largest K tried, and training stopped at 15 iterations), so a larger K or more iterations might do better. A train/validation/test split and several seeds would fix this.

Split and uncertainty. Our split is random within each user and ignores time, so the model may use future ratings to predict past ones; a temporal split would be closer to deployment, and the cold-start case (a movie with no training rating) falls back on biases only. We report point estimates (MAP); a Bayesian treatment, or a per-user choice of τ or K, would quantify uncertainty, which matters most for users with few ratings, where we observed the strongest overfitting.

Evaluation. The recommendation experiments are qualitative and use one profile. Ranking metrics such as Precision@k and Recall@k, and an online A/B test, would give stronger evidence.

Genre embeddings. The genre embeddings of Section 4.5 are fitted by a ridge regression on the movie vectors of a separate K=2 model, not by the full ALS of Section 4.4. This is the MAP solution for $\mathbf{W}$ given the movie vectors, so it isolates the effect of the genre tie; a joint training of all four blocks might give a different but related picture.

9. Conclusion

We built a rating-prediction system from a bias-only model to a latent factor model trained with ALS, and scaled it to 25 million ratings using CSR matrices and parallel compiled kernels. The best configuration ($K = 20$, $\tau = 10$) reaches a test RMSE of about 0.780, against 0.860 without latent factors. Overfitting grows with K and shrinks with τ and with the number of ratings per user, so users with few ratings are the main difficulty. A synthetic user who liked *The Fellowship of the Ring* receives sensible recommendations, once the weight of the movie bias is reduced. We also showed that the trait vectors can be projected into 2D to reveal genre structure, that genre vectors can be recovered by ridge regression and interpreted, and that the long movie vectors identify polarizing movies, a finding that is robust to overfitting.

Software and Data

All code is available at `https://github.com/AidanAcartis/Movie_Recommandation` and is shared with the GitHub user `upaq`.

References

Bennett, J. and Lanning, S. The Netflix Prize. In *Proceedings of KDD Cup and Workshop*, pp. 3–6, 2007.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009.

Zhou, Y., Wilkinson, D., Schreiber, R., and Pan, R. Large-scale parallel collaborative filtering for the Netflix Prize. In *Algorithmic Aspects in Information and Management (AAIM)*, volume 5034 of *Lecture Notes in Computer Science*, pp. 337–348. Springer, 2008.